人工智能与深度学习

强化学习（二）

华中科技大学 机械科学与工程学院
陈伟

2026

强化学习算法

- **我们详细介绍了深度 Q 学习（DQN）算法的执行细节**
- **我们介绍了 REINFORCE 算法和 Actor-Critic 算法的核心思想**

- **我们介绍了强化学习算法的分类**
  - 基于价值 vs 基于策略梯度
  - On-policy vs Off-policy

强化学习算法

- **我们继续介绍一些强化学习算法**
  - **DDQN**，A3C，Rainbow，**DDPG**，TRPO 与 PPO，**SAC**

- **关注这些算法的核心思想**
  - 部分算法的细节大家可以课后详细了解

Double Deep Q-Learning（DDQN）

回顾：Q学习

- Q 函数的价值迭代

$$Q^{i+1}(s,a) = R(s,a) + \mathbb{E}[\max_{a'} Q^i(s',a')],$$

其中 $s' \sim \mathbb{P}(\cdot|s,a)$。我们对其做一点改变，再设计基于数据采样的迭代算法，

$$Q^{i+1}(s,a) = (1-\eta)Q^{i+1}(s,a) + \eta R(s,a) + \eta \mathbb{E}[\max_{a'} Q^i(s',a')],$$



$$Q(s,a) \leftarrow (1-\eta)Q(s,a) + \eta[R(s,a,s') + \max_{a'} Q(s',a')],$$

更新

回顾：Q学习

- $$Q(s,a) \leftarrow (1-\eta)Q(s,a) + \eta[R(s,a,s') + \max_{a'} Q(s',a')],$$

  这里需要进行一步仿真（需用到一个 (s,a,r,s') 样本）

- 上式也可以写成如下等价形式

$$Q(s,a) \leftarrow Q(s,a) + \eta[R(s,a,s') + \max_{a'} Q(s',a') - Q(s,a)],$$

学习率　　根据差异，更新Q函数

回顾：深度 Q 学习

- $$Q^*(s,a) = \mathbb{E}[r + \gamma \max_{a'} Q^*(s',a')],$$

- 定义 DQN 的损失函数

$$L_i(\theta_i) = \mathbb{E}\left[(y_i - Q(s,a;\theta_i))^2\right],$$

其中 $y_i = \mathbb{E}[r + \gamma \max_{a'} Q(s',a';\theta_{i-1})]$

回顾：深度Q学习

神经网络训练需要知道损失函数 L_i 关于网络参数 θ 的梯度

$$\nabla_{\theta_i} L_i = \mathbb{E}\left[\left(r+\gamma \max_{a'} Q(s', a'; \theta_{i-1}) - Q(s, a; \theta_i)\right) \underbrace{\nabla_{\theta_i} Q(s, a; \theta_i)}\right]$$

神经网络的输出关于其参数的梯度，通过一般网络的的反向传播算法可计算

- 有了参数梯度信息后，便可用随机梯度下降的方法训练神经网络了

$$\theta_{i+1} = \theta_i - \eta \nabla_{\theta_i} L_i$$

- 每次训练，需要使用一批（s,a,r,s’）样本

Q 学习的一个缺陷：过估计（overestimation）

- Q 学习会过高地估计某些状态-动作组合的 Q 值

- 高估是“不均匀的”，导致无法得到最优策略

Q 学习的一个缺陷：过估计（overestimation）

- **对 Q 学习过估计的一种解释：对一组样本先求最大值再求平均，通常并先求平均再求最大值要大**

$$\mathbb{E}(\max(X_1, X_2, ...)) \geq \max(\mathbb{E}X_1, \mathbb{E}X_2, ...)$$

- 理想情况下（模型已知）Q 函数的价值迭代

$$Q^{i+1}(s,a) = R(s,a) + \mathbb{E}[\max_{a'} Q^i(s',a')],$$

实际中的 Q 学习（下标$_s$代表可理解为某种样本）

$$Q_s^{i+1}(s,a) \leftarrow Q_s^i(s,a) + \eta[r_s + \max_{a'} Q_s^i(s'_s,a') - Q_s^i(s,a)]$$

Double Q-Learning

- **Double：保存两套 Q 函数，在更新中互相使用对方**

- **使用不同的样本更新两套 Q 函数**

Double Q-Learning

- Single Q-learning

$$Q(s,a) \leftarrow Q(s,a) + \eta[r + Q(s', a^*) - Q(s,a)],$$

其中 $a^* = \arg\max_{a'} Q(s', a')$

- Double Q-learning

$$Q^A(s,a) \leftarrow Q^A(s,a) + \eta[r + Q^B(s', a^A) - Q^A(s,a)]$$
$$Q^B(s,a) \leftarrow Q^B(s,a) + \eta[r + Q^A(s', a^B) - Q^B(s,a)]$$

其中 $a^A = \arg\max_{a'} Q^A(s', a')$, $a^B = \arg\max_{a'} Q^B(s', a')$,
两个 Q 函数用不同的样本训练

Double Q-Learning 直接用于 Deep Q-Learning

- Single DQN 的损失函数

$$L_i(\theta_i) = (y_i - Q(s, a; \theta_i))^2,$$

其中

$$y_i = r + \gamma Q(s', a^*; \theta_{i-1}),$$

$a^* = \arg\max'_a Q(s', a', \theta_{i-1})$, y_i 有时也被成为学习的目标（target）

Double Q-Learning：使用两个平等的 Q 网络

- 我们可以使用含有两个 Q 网络的 Deep Q-Learning，以实现 Double Q-Learning：损失函数不变，而目标

$$y_i^A = r + \gamma Q^B(s', a^*; \theta_{i-1}^B),$$

其中 $a^* = \arg\max'_a Q^A(s', a', \theta_{i-1}^A)$;

$$y_i^B = r + \gamma Q^A(s', a^*; \theta_{i-1}^A),$$

其中 $a^* = \arg\max'_a Q^B(s', a', \theta_{i-1}^B)$

缺点：效率不高

使用目标网络的 DQN

- 我们首先介绍使用目标网络（target network）的 DQN。

  损失函数

$$L_i(\theta_i) = (y_i - Q(s, a; \theta_i))^2,$$

  其中

$$y_i = r + \gamma Q(s', a^*; \theta^-),$$

$$a^* = \arg\max'_a Q(s', a', \theta^-).$$

  $Q(s, a; \theta^-)$ 被称为目标网络；每隔 τ 步，我们就令 $\theta^- = \theta_i$ （即上式中的 θ^- 其实就是几步之前的 θ_i，θ^- 的更新不如 θ_i 频繁）

Q 网络更新的基本思路

- Compute a target y



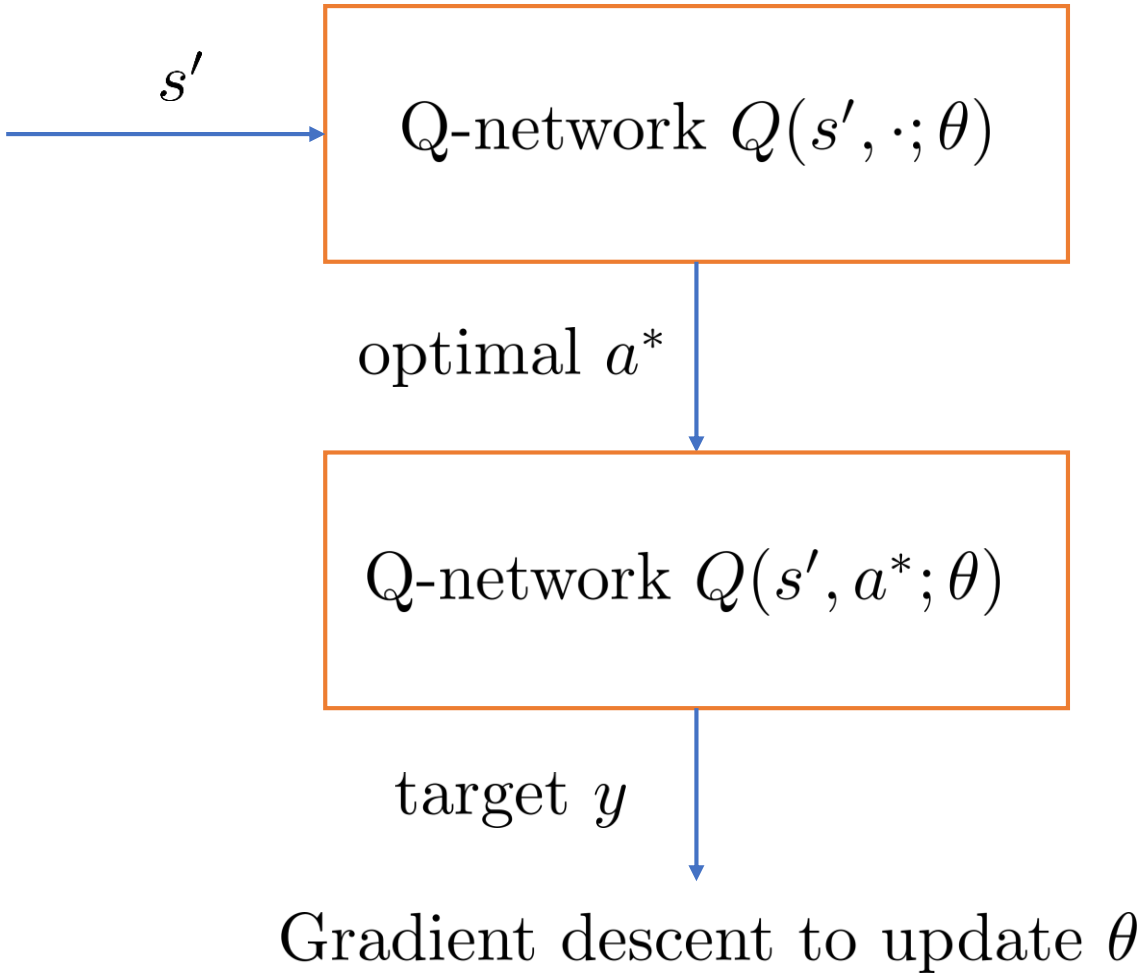
Define a loss $L = (y - Q(s, a; \theta))^2$



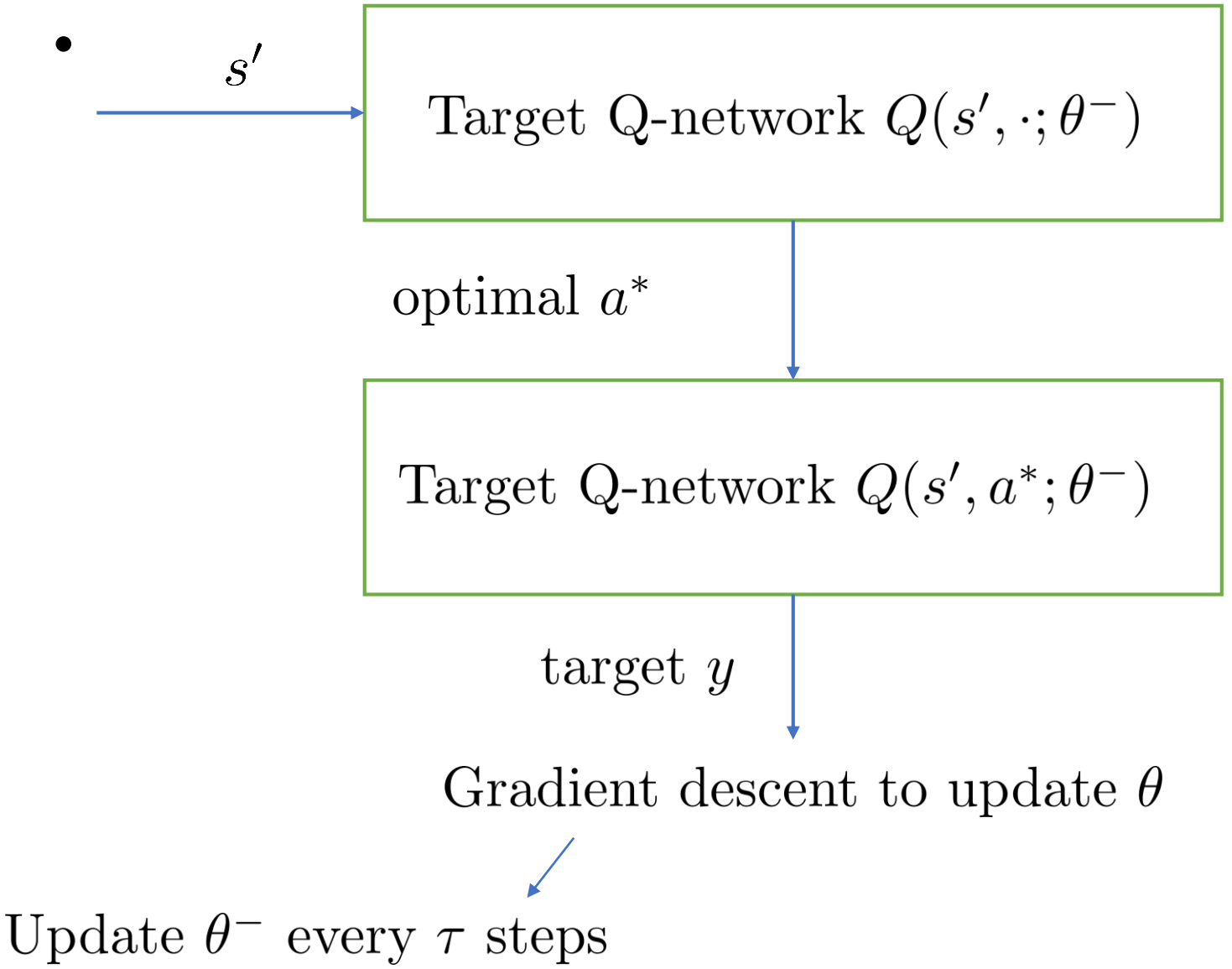
Minimize the loss using gradient descent

$$\theta \leftarrow \theta - \eta \nabla_\theta L$$

回顾：不使用目标网络的 DQN



使用目标网络的 DQN


s'
Target Q-network $Q(s', \cdot; \theta^-)$
optimal a^*
Target Q-network $Q(s', a^*; \theta^-)$
target y
Gradient descent to update θ
Update θ^- every τ steps

DDQN：部分使用目标网络

- DDQN 的损失函数

$$L_i(\theta_i) = (y_i - Q(s, a; \theta_i))^2,$$

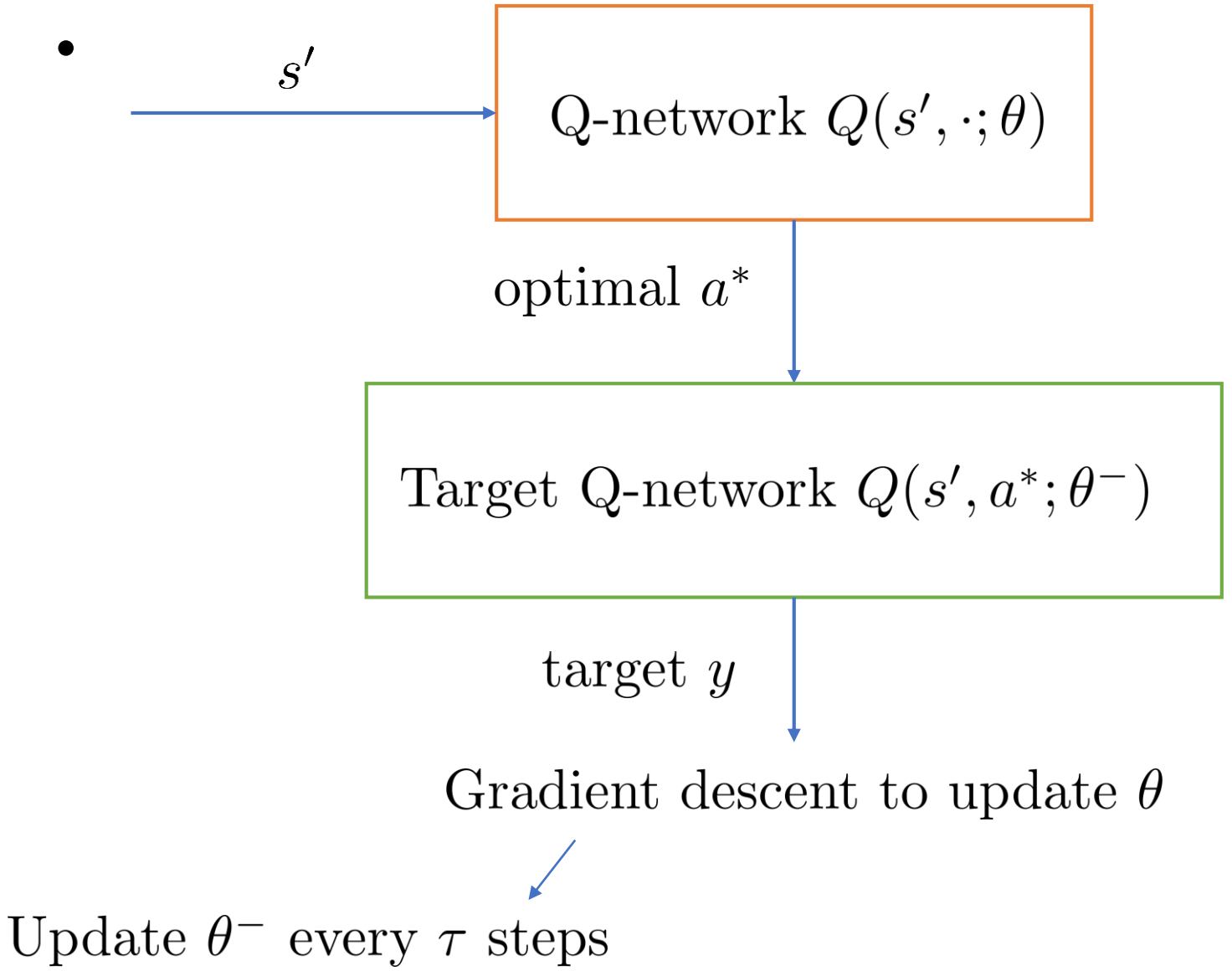
其中

$$y_i = r + \gamma Q(s', a^*; \theta^-),$$

$a^* = \arg\max'_a Q(s', a', \theta_{i-1})$.

$Q(s, a; \theta^-)$ 被称为目标网络；每隔 τ 步，我们就令 $\theta^- = \theta_i$

DDQN：部分使用目标网络



DDQN

- **相对于原始的 DQN 和使用目标网络的 DQN，DDQN 只进行了很小的改进，但能取得明显的效果**

Asynchronous Advantage Actor Critic (A3C)

A3C：Asynchronous

- **A3C 的核心思想是使用多个 Actor-Critic 并行训练，以提升学习的效率**

- **Asynchronous：多个 agents 异步训练**

A3C：Advantage

- 回顾：策略的迭代中，我们使用如下公式估算策略梯度

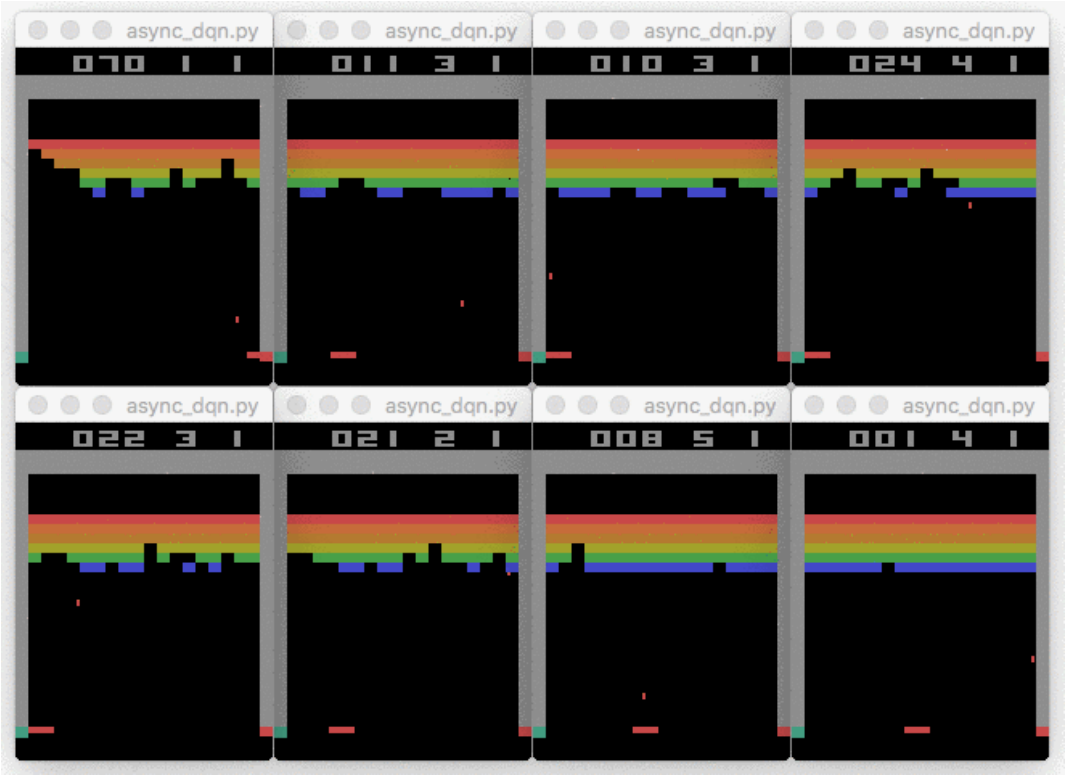
$$\nabla_\theta J(\theta) \approx [G_t - b(s_t)] \nabla_\theta \log \pi_\theta(a_t|s_t),$$

其中的 $G_t - b(s_t)$ 可以用 $Q^\pi(s_t, a_t) - V^\pi(s_t)$ 替代，因为 G_t (奖励之和)近似 $Q^\pi(s, a)$，$V^\pi(s_t)$ 可当作基线 $b(s_t)$

$A(s, a) = Q^\pi(s, a) - V^\pi(s)$ 也被称为“优势”（Advantage）

A3C：Actor-Critic

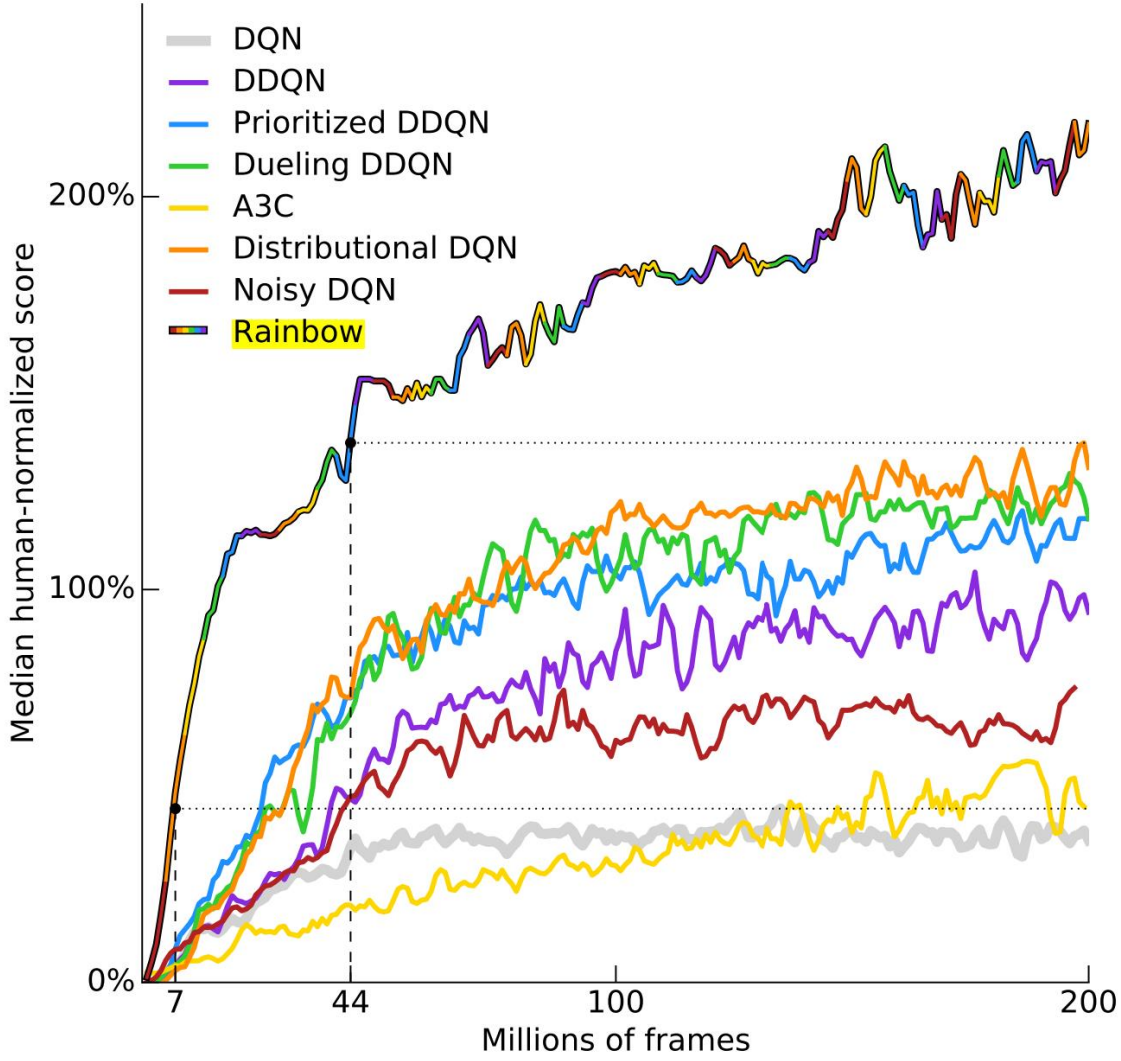
- **A3C 基于 Actor-Critic 结构，即同时需要更新策略神经网络和 Q 神经网络**
- **同时，采用多个 agents 异步训练**



Rainbow

Rainbow 与 基础 DQN 等的对比

- Median human-normalized performance across 57 Atari games



Rainbow：综合对 DQN 的多项改进

- **Double Deep Q-learning**
- **Prioritized replay**
- **Dueling networks**
- **Multi-step learning**
- **Distributional RL**
- **Noisy nets**

Rainbow： Double Deep Q-learning

- DDQN 的损失函数

$$L_i(\theta_i) = (y_i - Q(s, a; \theta_i))^2,$$

其中

$$y_i = r + \gamma Q(s', a^*; \theta^-),$$

$$a^* = \arg\max'_a Q(s', a', \theta_{i-1}).$$

Rainbow： Prioritized replay

- 基础 DQN 从经验回放的数据中随机均匀地抽样，用于训练

  Prioritized replay 在抽样时，对于目前“更需要重点学习”的样本，给与更大的概率

$$\mathbb{P} \propto \left| r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right|^{\omega}$$

Rainbow： Dueling networks

- Dueling networks 指的是一种适用于 Q 函数神经网络的结构；对于 Atari 游戏，共用同样的卷积层处理前端信息，然后分别对价值 $V(s)$ 和优势 $Q(s,a)-V(s)$ 进行处理

Rainbow： Multi-step learning

- 在单步的学习中，我们希望最小化以下这样的损失

$$[r + \gamma \max_{a'} Q(s', a'; \theta) - Q(s, a; \theta)]^2$$

在多步的学习中，我们最小化以下损失

$$[\sum_{i=1}^{n} r_i + \gamma^n \max_{a'} Q(s', a'; \theta) - Q(s, a; \theta)]^2,$$

其中 s' 符合 n 步之后状态的概率分布

Rainbow： Distributional RL 与 Noisy nets

- 在之前的学习中，我们仅考虑了回报的期望

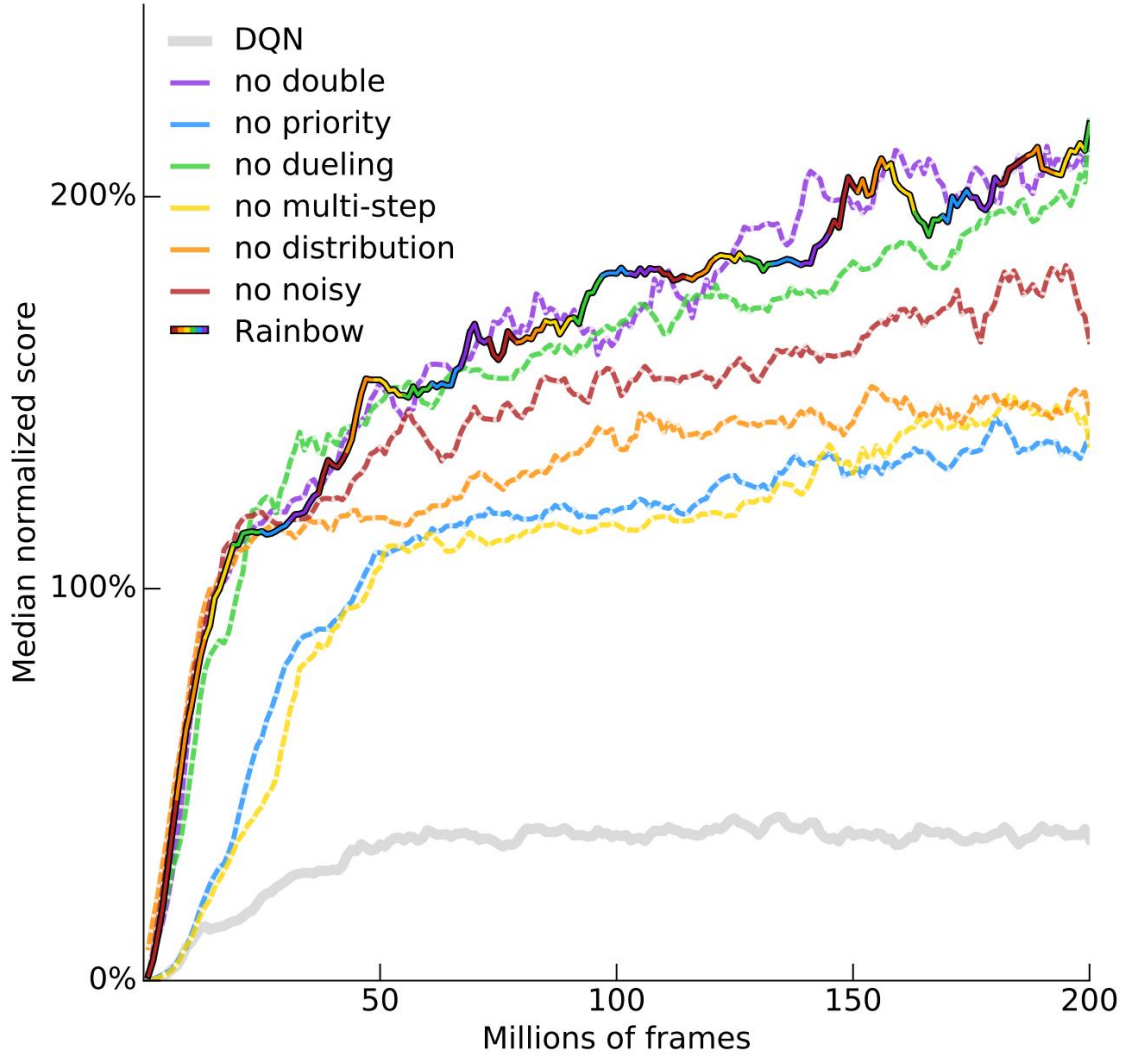
  在 distributional RL 中，我们希望考虑回报的概率分布

- Noisy Nets 针对的是强化学习的探索问题

  通过在网络中结合一个确定性的输出和一个带噪音的输出，可以得到更好的探索动作；同时这些探索动作会随着学习慢慢减少

Rainbow 与 去掉某元素的 Rainbow 对比

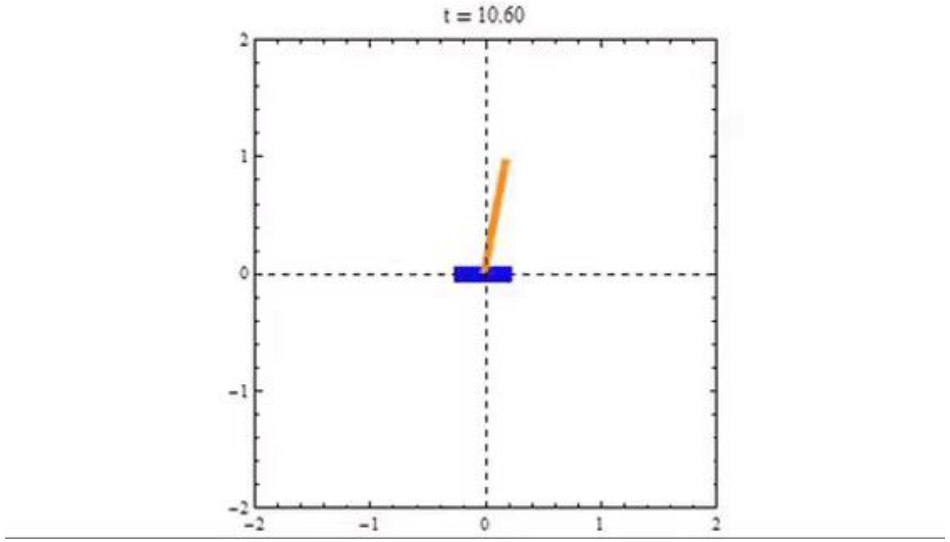
- Median human-normalized performance across 57 Atari games



Deep Deterministic Policy Gradient (DDPG)

连续控制系统

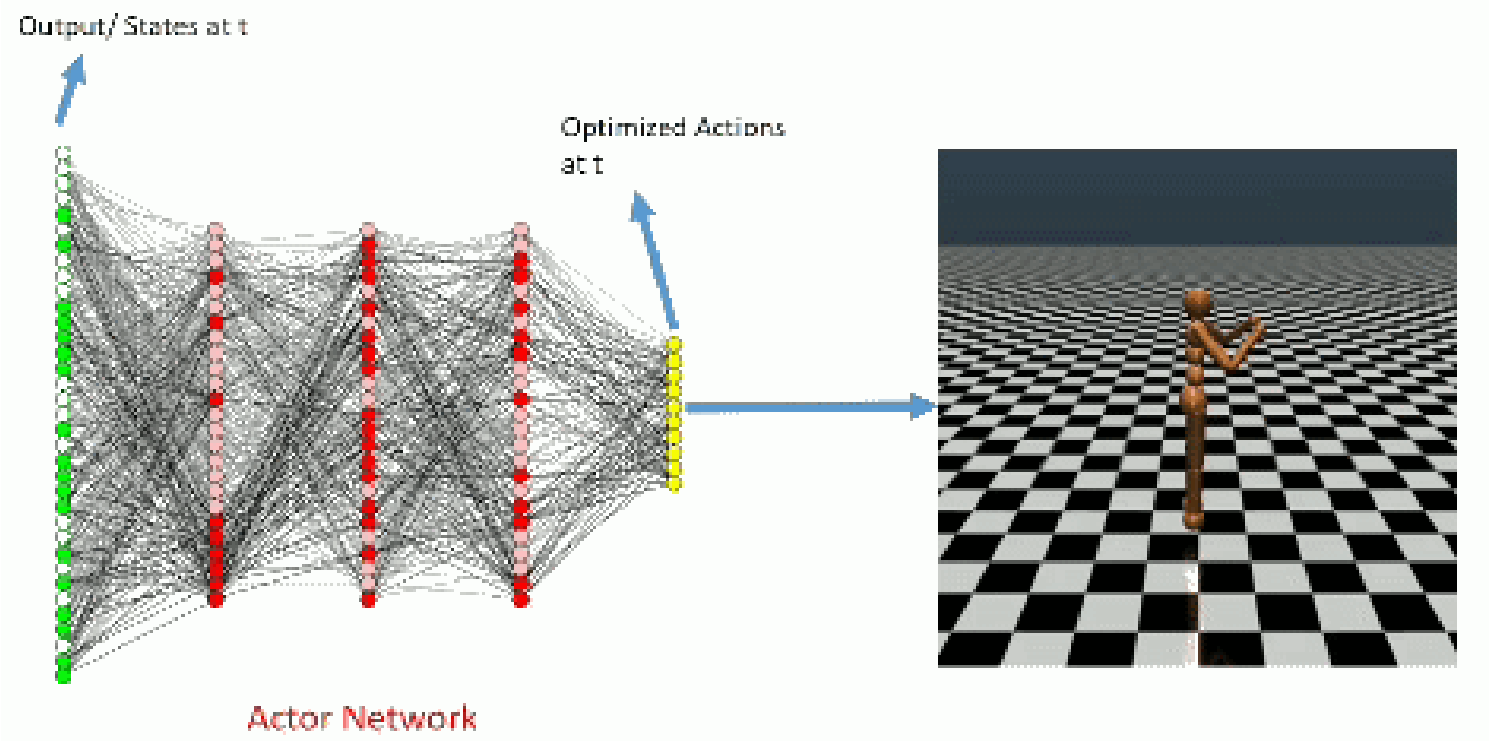
- 将连续的控制系统描述为马尔可夫决策过程，通常需要做一些近似





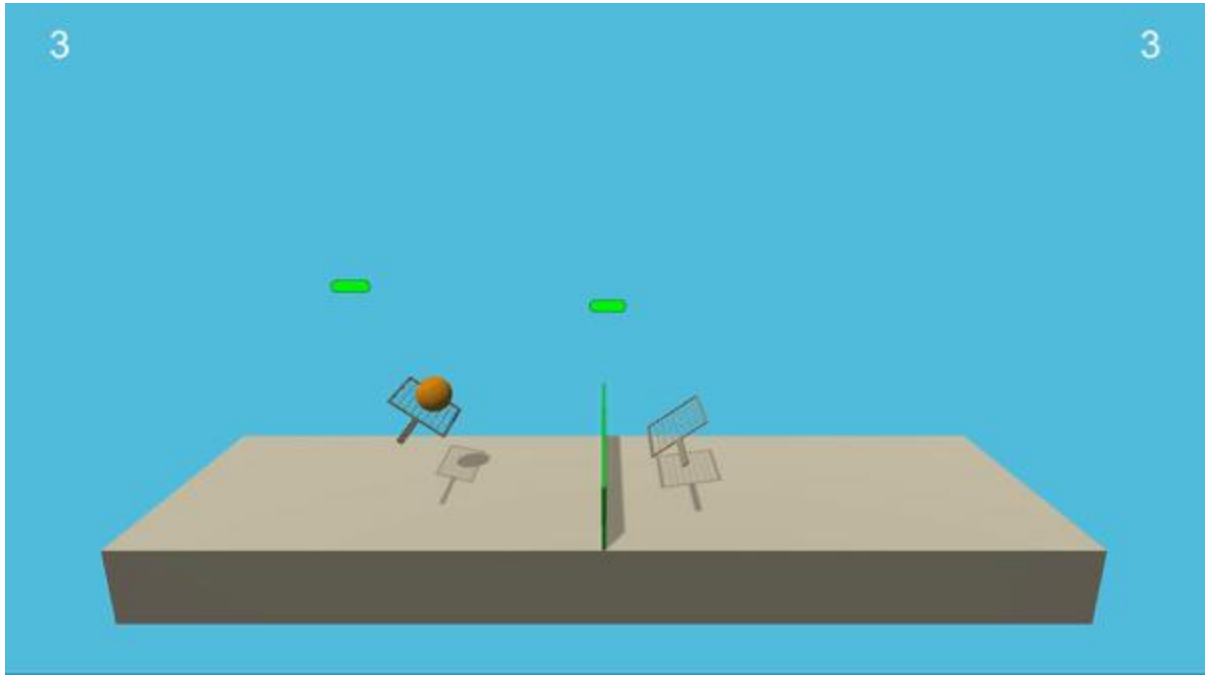
连续控制系统

- **DDPG 算法在连续系统的控制上有较好表现**
  - Deterministic Policy： 确定性的策略，适合控制问题



DDPG 应用

-



DDPG 的 Actor-Critic 结构

- DDPG 使用两个网络：Q 神经网络 $Q(s,a;\theta^Q)$ 和策略网络 $\pi(s;\theta^\pi)$

  与 DDQN 类似，DDPG 还使用两套目标参数 $\theta^{Q'}$ 与 $\theta^{\pi'}$；与 DDQN 不完全相同的是，$\theta^{Q'}$ 与 $\theta^{\pi'}$ 不是采用“低频率更新”的形式靠近 θ^Q 与 θ^π，而是采用一阶滤波更新的方式（soft update），即每步中

$$\theta^{Q'} \leftarrow (1-\tau)\theta^{Q'} + \tau\theta^{Q}$$

$$\theta^{\pi'} \leftarrow (1-\tau)\theta^{\pi} + \tau\theta^{\pi}$$

DDPG 的 Q 网络更新

- DDPG 的损失函数

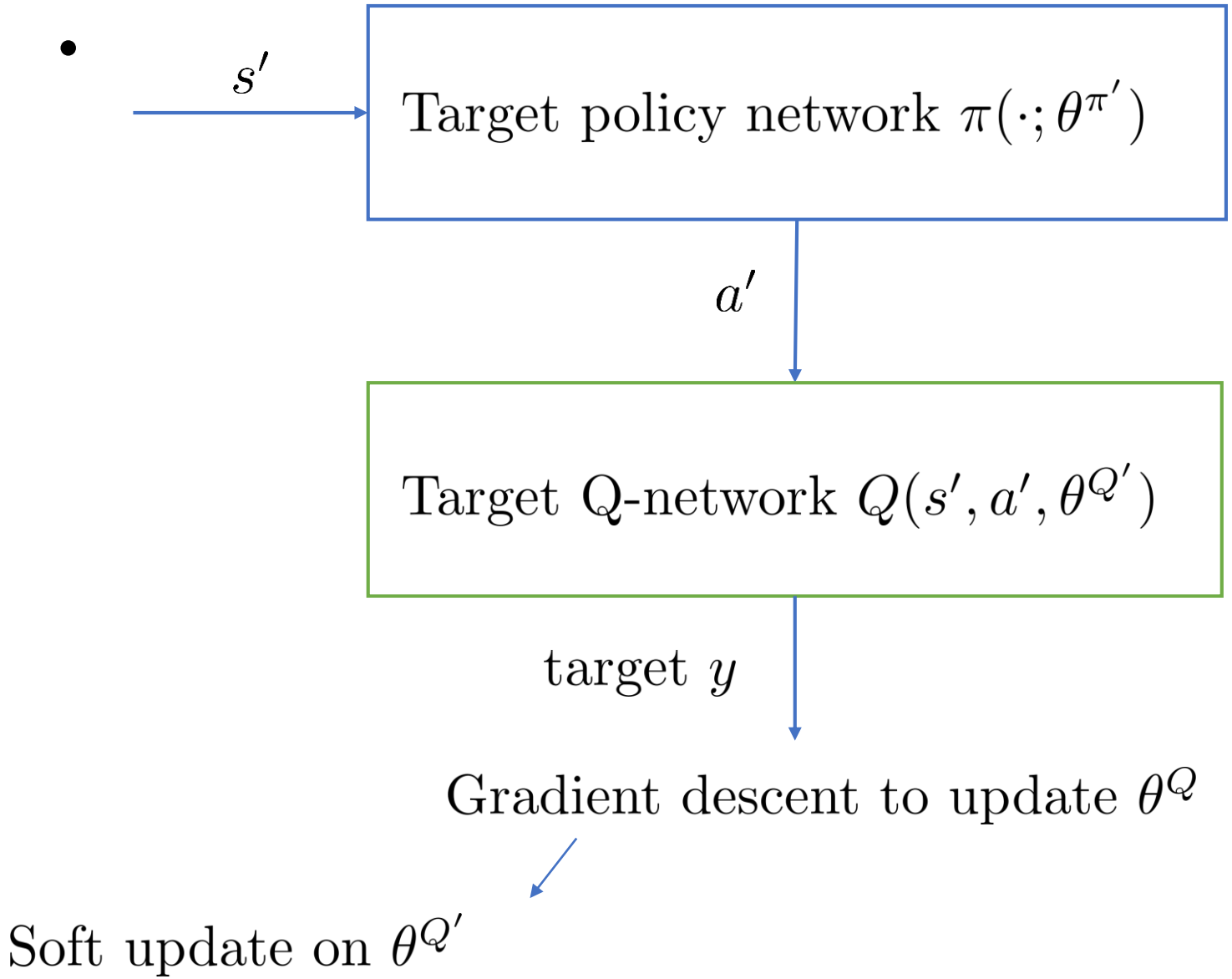
$$L_i(\theta^Q) = (y_i - Q(s, a; \theta^Q))^2,$$

其中

$$y_i = r + \gamma Q(s', \pi(s'; \theta^{\pi'}); \theta^{Q'})$$

使用随机梯度下降，用损失函数关于 θ^Q 的梯度更新 θ^Q

DDPG 的 Q 网络更新


s'
Target policy network $\pi(\cdot;\theta^{\pi'})$
a'
Target Q-network $Q(s', a', \theta^{Q'})$
target y
Gradient descent to update θ^{Q}
Soft update on $\theta^{Q'}$

DDPG 的策略更新

- DDPG 的策略梯度

$$\nabla_{\theta^\pi} J \approx \frac{1}{N} \sum \nabla_a Q(s, a; \theta^Q)|_{a=\pi(s;\theta^\pi)} \nabla_{\theta^\pi} \pi(s; \theta^\pi)$$

用 J 关于 θ^π 的梯度更新 θ^π

DDPG 算法流程

- **Algorithm 1** DDPG algorithm

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ.
Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q$, $\theta^{\mu'} \leftarrow \theta^\mu$
Initialize replay buffer R
for episode = 1, M **do**
  Initialize a random process $\mathcal{N}$ for action exploration
  Receive initial observation state s_1
  for t = 1, T **do**
    Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise
    Execute action a_t and observe reward r_t and observe new state s_{t+1}
    Store transition (s_t, a_t, r_t, s_{t+1}) in R
    Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R
    Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$
    Update critic by minimizing the loss: $L = \frac{1}{N}\sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$
    Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N}\sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu}\mu(s|\theta^\mu)|_{s_i}$$

    Update the target networks:

$$\theta^{Q'} \leftarrow \tau\theta^Q + (1-\tau)\theta^{Q'}$$
$$\theta^{\mu'} \leftarrow \tau\theta^\mu + (1-\tau)\theta^{\mu'}$$

  end for
end for

DDPG 算法流程

- 初始化：Q 网络、策略网络及它们的目标网络；样本库

Algorithm 1 DDPG algorithm

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ.
Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q$, $\theta^{\mu'} \leftarrow \theta^\mu$
Initialize replay buffer R
for episode = 1, M **do**
  Initialize a random process $\mathcal{N}$ for action exploration
  Receive initial observation state s_1
  for t = 1, T **do**
    Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise
    Execute action a_t and observe reward r_t and observe new state s_{t+1}
    Store transition (s_t, a_t, r_t, s_{t+1}) in R
    Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R
    Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$
    Update critic by minimizing the loss: $L = \frac{1}{N}\sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$
    Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i}$$

    Update the target networks:

$$\theta^{Q'} \leftarrow \tau\theta^Q + (1-\tau)\theta^{Q'}$$
$$\theta^{\mu'} \leftarrow \tau\theta^\mu + (1-\tau)\theta^{\mu'}$$

  end for
end for

DDPG 算法流程

- 每回合内，初始化动作噪音（用于探索），和状态初值

Algorithm 1 DDPG algorithm

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ.
Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q$, $\theta^{\mu'} \leftarrow \theta^\mu$
Initialize replay buffer R
for episode = 1, M **do**
  Initialize a random process $\mathcal{N}$ for action exploration
  Receive initial observation state s_1
  for t = 1, T **do**
    Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise
    Execute action a_t and observe reward r_t and observe new state s_{t+1}
    Store transition (s_t, a_t, r_t, s_{t+1}) in R
    Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R
    Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$
    Update critic by minimizing the loss: $L = \frac{1}{N}\sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$
    Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i}$$

    Update the target networks:

$$\theta^{Q'} \leftarrow \tau\theta^Q + (1-\tau)\theta^{Q'}$$
$$\theta^{\mu'} \leftarrow \tau\theta^\mu + (1-\tau)\theta^{\mu'}$$

  end for
end for

DDPG 算法流程

- 随机探索：探索的动作 = 已知最优动作 + 噪音

Algorithm 1 DDPG algorithm

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ.
Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q$, $\theta^{\mu'} \leftarrow \theta^\mu$
Initialize replay buffer R
for episode = 1, M **do**
  Initialize a random process $\mathcal{N}$ for action exploration
  Receive initial observation state s_1
  for t = 1, T **do**
    Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise
    Execute action a_t and observe reward r_t and observe new state s_{t+1}
    Store transition (s_t, a_t, r_t, s_{t+1}) in R
    Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R
    Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$
    Update critic by minimizing the loss: $L = \frac{1}{N}\sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$
    Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N}\sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu}\mu(s|\theta^\mu)|_{s_i}$$

    Update the target networks:

$$\theta^{Q'} \leftarrow \tau\theta^Q + (1-\tau)\theta^{Q'}$$
$$\theta^{\mu'} \leftarrow \tau\theta^\mu + (1-\tau)\theta^{\mu'}$$

  end for
end for

DDPG 算法流程

- 选取 mini-batch（经验回放）

Algorithm 1 DDPG algorithm

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ.
Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q$, $\theta^{\mu'} \leftarrow \theta^\mu$
Initialize replay buffer R
for episode = 1, M **do**
  Initialize a random process $\mathcal{N}$ for action exploration
  Receive initial observation state s_1
  for t = 1, T **do**
    Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise
    Execute action a_t and observe reward r_t and observe new state s_{t+1}
    Store transition (s_t, a_t, r_t, s_{t+1}) in R
    Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R
    Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$
    Update critic by minimizing the loss: $L = \frac{1}{N}\sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$
    Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i}$$

    Update the target networks:

$$\theta^{Q'} \leftarrow \tau\theta^Q + (1-\tau)\theta^{Q'}$$
$$\theta^{\mu'} \leftarrow \tau\theta^\mu + (1-\tau)\theta^{\mu'}$$

  end for
end for

DDPG 算法流程

- 更新 Q 网络：计算 Q 网络更新的目标及损失函数；根据损失函数的梯度，进行梯度下降

Algorithm 1 DDPG algorithm

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ.
Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q$, $\theta^{\mu'} \leftarrow \theta^\mu$
Initialize replay buffer R
for episode = 1, M **do**
  Initialize a random process $\mathcal{N}$ for action exploration
  Receive initial observation state s_1
  for t = 1, T **do**
    Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise
    Execute action a_t and observe reward r_t and observe new state s_{t+1}
    Store transition (s_t, a_t, r_t, s_{t+1}) in R
    Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R
    Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$
    Update critic by minimizing the loss: $L = \frac{1}{N}\sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$
    Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N}\sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu}\mu(s|\theta^\mu)|_{s_i}$$

    Update the target networks:

$$\theta^{Q'} \leftarrow \tau\theta^Q + (1-\tau)\theta^{Q'}$$
$$\theta^{\mu'} \leftarrow \tau\theta^\mu + (1-\tau)\theta^{\mu'}$$

  end for
end for

DDPG 算法流程

- 更新策略网络：计算策略梯度，进行梯度下降

Algorithm 1 DDPG algorithm

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ.
Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q$, $\theta^{\mu'} \leftarrow \theta^\mu$
Initialize replay buffer R
for episode = 1, M **do**
  Initialize a random process $\mathcal{N}$ for action exploration
  Receive initial observation state s_1
  for t = 1, T **do**
    Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise
    Execute action a_t and observe reward r_t and observe new state s_{t+1}
    Store transition (s_t, a_t, r_t, s_{t+1}) in R
    Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R
    Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$
    Update critic by minimizing the loss: $L = \frac{1}{N}\sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$
    Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N}\sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu}\mu(s|\theta^\mu)|_{s_i}$$

    Update the target networks:

$$\theta^{Q'} \leftarrow \tau\theta^Q + (1-\tau)\theta^{Q'}$$
$$\theta^{\mu'} \leftarrow \tau\theta^\mu + (1-\tau)\theta^{\mu'}$$

  end for
end for

DDPG 算法流程

- Soft update Q 目标网络和策略目标网络

Algorithm 1 DDPG algorithm

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ.
Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q$, $\theta^{\mu'} \leftarrow \theta^\mu$
Initialize replay buffer R
for episode = 1, M **do**
  Initialize a random process $\mathcal{N}$ for action exploration
  Receive initial observation state s_1
  for t = 1, T **do**
    Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise
    Execute action a_t and observe reward r_t and observe new state s_{t+1}
    Store transition (s_t, a_t, r_t, s_{t+1}) in R
    Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R
    Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$
    Update critic by minimizing the loss: $L = \frac{1}{N}\sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$
    Update the actor policy using the sampled policy gradient:

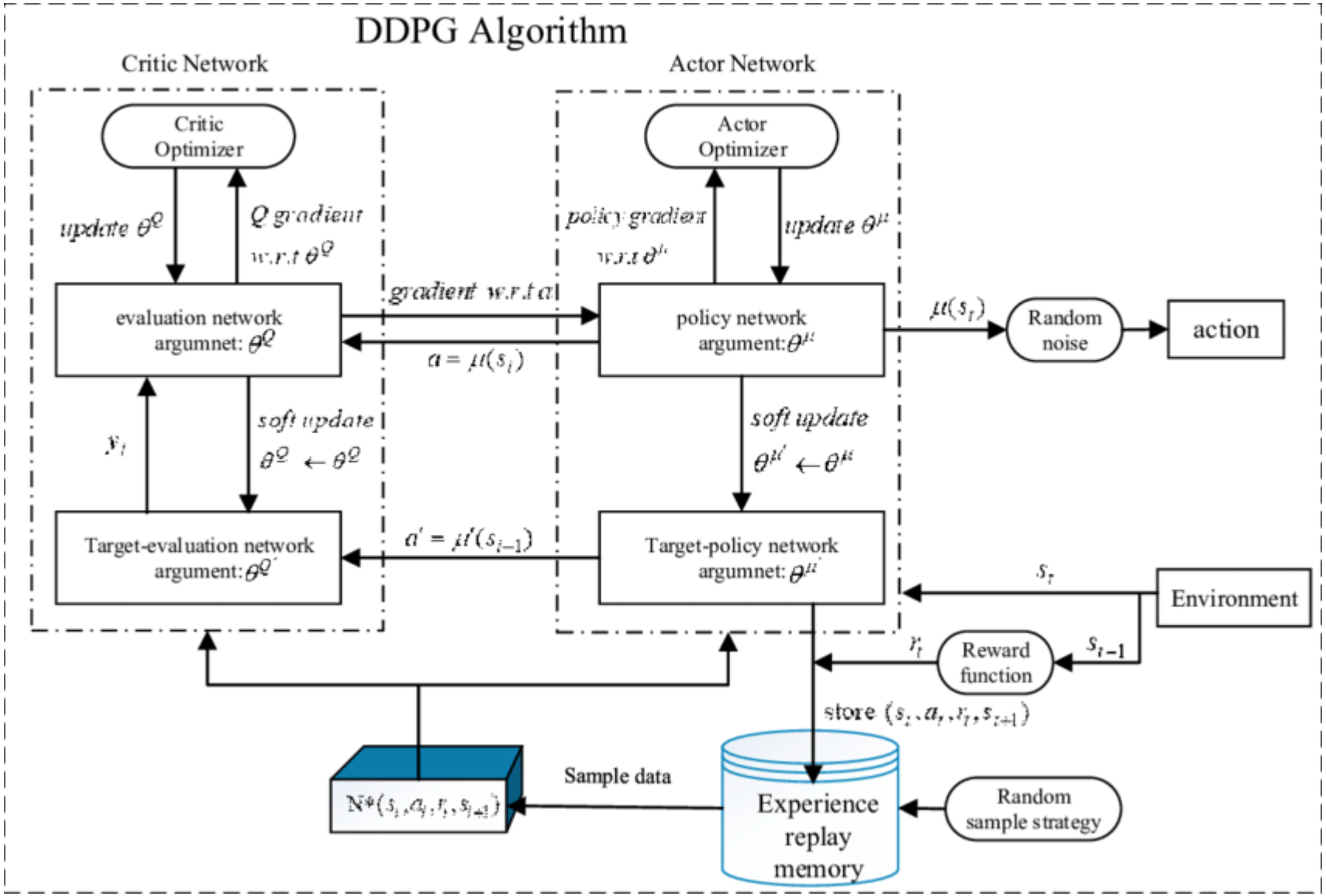
$$\nabla_{\theta^\mu} J \approx \frac{1}{N}\sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu}\mu(s|\theta^\mu)|_{s_i}$$

    Update the target networks:

$$\theta^{Q'} \leftarrow \tau\theta^Q + (1-\tau)\theta^{Q'}$$
$$\theta^{\mu'} \leftarrow \tau\theta^\mu + (1-\tau)\theta^{\mu'}$$

  end for
end for

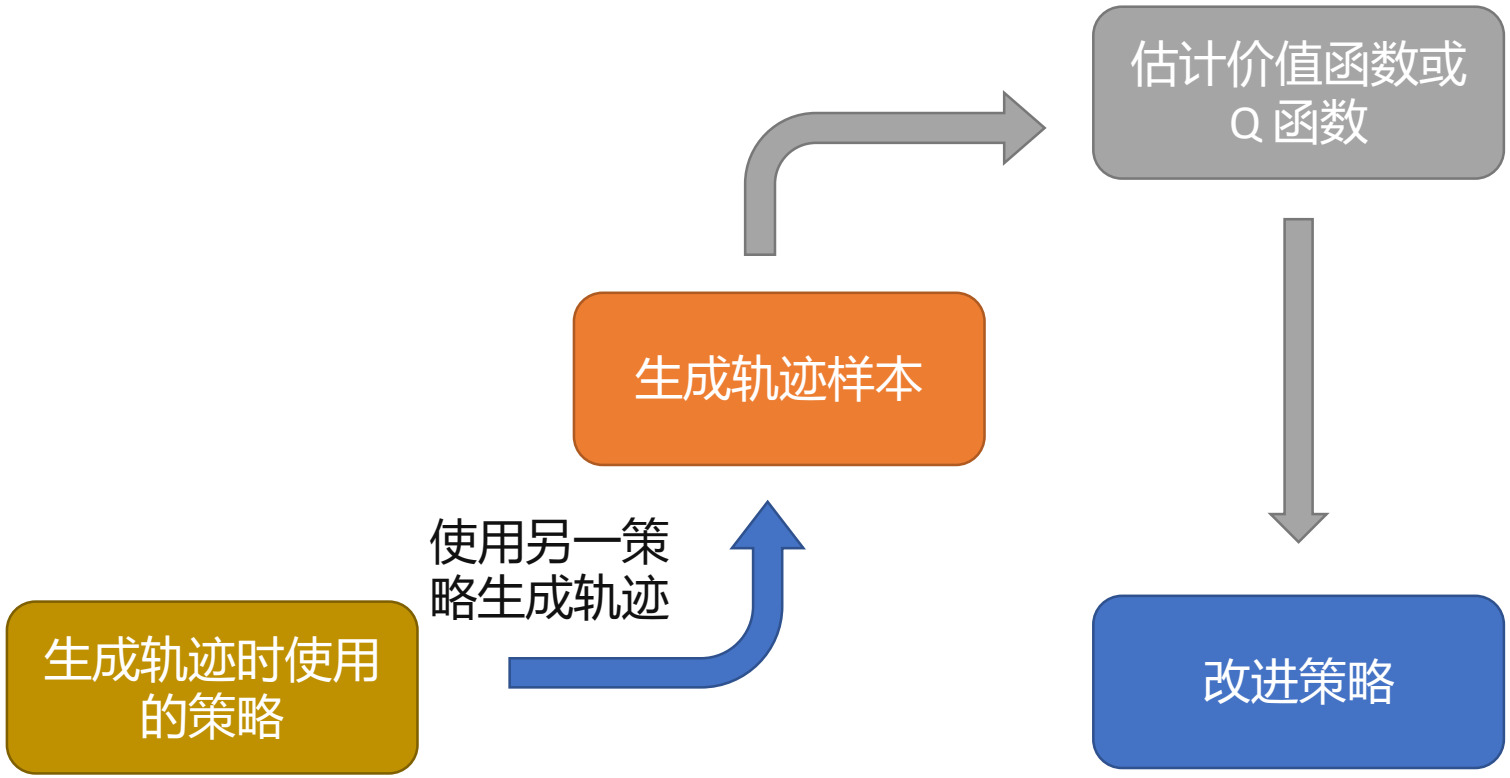
DDPG 算法流程

- 
DDPG Algorithm
Critic Network
Actor Network
Critic Optimizer
Actor Optimizer
update θ^Q
Q gradient w.r.t θ^Q
policy gradient w.r.t θ^μ
update θ^μ
evaluation network argumnet: θ^Q
gradient w.r.t a
$a = \mu(s_t)$
policy network argument: θ^μ
$\mu(s_T)$
Random noise
action
y_t
soft update $\theta^{Q'} \leftarrow \theta^Q$
soft update $\theta^{\mu'} \leftarrow \theta^\mu$
Target-evaluation network argument: $\theta^{Q'}$
$a' = \mu'(s_{i-1})$
Target-policy network argumnet: $\theta^{\mu'}$
s_t
Environment
r_t
Reward function
s_{t-1}
store (s_t, a_t, r_t, s_{t+1})
Sample data
Experience replay memory
Random sample strategy

Off-Policy 方法中的重要性采样

Off-Policy 方法

- **基于 off-policy 的强化学习方法包含 TRPO 和 PPO 等**
- **Off-policy 方法中通常都需要用到重要性采样**



重要性采样（importance sampling）

- 我们希望估计某策略 π 的价值函数 V_π 或者 Q 函数 Q_π

  以价值函数为例，

  $$V_\pi(s) = \mathbb{E}G_t$$

  其中 $G_t = \sum_{t \geq 0} r_t$ 是从状态 s 开始的轨迹的奖励

  假设我们的轨迹样本也都来自策略 π，那么我们直接使用多个 G_t 样本，求平均，即可得到 $V_\pi(s)$ 的估计值

重要性采样（importance sampling）

- 假设我们的轨迹样本不是来自策略 π，而是来自另外一个策略 b

  此时（策略 b 下）样本轨迹的概率分布，与策略 π 下样本轨迹的概率分布是不同的，因此不能通过直接求平均，得到我们想要的 $\mathbb{E}G_t$

  我们需要知道，两个概率分布的关系

重要性采样（importance sampling）

- 假设初值 s_0 是固定的，那么策略 π 下轨迹 $a_0, s_1, a_1, s_2, \ldots, s_N$ 的概率

$$\mathbb{P}(a_0, s_1, a_1, s_2, \ldots, s_N | \pi)$$

$$= \pi(a_0|s_0)\mathbb{P}(s_1|s_0, a_0)\pi(a_1|s_1)\mathbb{P}(s_2|s_1, a_1)\ldots\mathbb{P}(s_N|s_{N-1}, a_{N-1})$$

$$= \prod_{t=0}^{N-1} \pi(a_k|s_k)\underbrace{\mathbb{P}(s_{k+1}|s_k, a_k)}$$

这些概率的值是未知的

重要性采样（importance sampling）

- 重要性采样比率（importance-sampling ratio）是两个策略下该轨迹出现的概率之比

$$\rho = \frac{\prod_{t=0}^{N-1} \pi(a_k|s_k)\mathbb{P}(s_{k+1}|s_k,a_k)}{\prod_{t=0}^{N-1} b(a_k|s_k)\mathbb{P}(s_{k+1}|s_k,a_k)} = \prod_{t=0}^{N-1} \frac{\pi(a_k|s_k)}{b(a_k|s_k)}$$

未知项在分子与分母中消掉了

因此重要性采样比率仅由两个策略决定

重要性采样（importance sampling）

- 假如我们在策略 b 下取得了 K 条轨迹的奖励样本 $\{G_k\}$；每个样本对应的重要性采样比率 $\{\rho_k\}$ 可以用策略 b 和 π 算出

  若要估计 V_b，我们可以用

$$V_b = \frac{\sum_k G_k}{K}$$

  若要估计 V_π，我们可以使用 ordinary importance sampling

$$V_\pi = \frac{\sum_k \rho_k G_k}{K}$$

重要性采样（importance sampling）

- 除了 ordinary importance sampling，我们还可以用 weighted importance sampling 来估计 V_π

$$V_\pi = \frac{\sum_k \rho_k G_k}{\sum_k \rho_k}$$

Weighted importance sampling 的方法是有偏的（biased），即其这样计算得到的结果的期望，并不等于我们想要计算的 V_π；但它的方差会小于无偏的（unbiased）ordinary importance sampling 方法

重要性采样（importance sampling）

- **许多策略梯度相关的算法，都会用到 off-policy 学习，因此会用到重要性采样的思想**

Trust Region Policy Optimization（TRPO）与（Proximal Policy Optimization）PPO

TRPO 与 PPO

- **TRPO 与 PPO 都是针对策略梯度方法难训练的问题**
  - 策略梯度估算不准确（“随机梯度”采样带来的问题）
  - 最佳的步长难以确定
    - 步长过长：单步没有改进
    - 步长过短：收敛太慢

- **对于策略梯度方法最直接的改进思路：使用二阶导数信息**
  - 但二阶信息会使计算量极具增大，完全不实用

TRPO 与 PPO

- **TRPO 与 PPO 都是通过某些近似，达到比仅使用一阶导数信息更好的效果**
  - PPO 可以看作是基于 TRPO 的改进方法

- **TRPO 算法实现比较复杂，PPO 算法相对简单一些**
  - PPO 提出了新的损失函数

Algorithm 1 PPO, Actor-Critic Style

for iteration=$1, 2, \ldots$ **do**
  for actor=$1, 2, \ldots, N$ **do**
    Run policy $\pi_{\theta_{\text{old}}}$ in environment for T timesteps
    Compute advantage estimates $\hat{A}_1, \ldots, \hat{A}_T$
  end for
  Optimize surrogate L wrt θ, with K epochs and minibatch size $M \leq NT$
  $\theta_{\text{old}} \leftarrow \theta$
end for

Soft Actor-Critic（SAC）

最大化熵

- “Soft”指的是策略不再简单追求最大化奖励 $\mathbb{E}\sum_{t\geq 0} r_t$，而是考虑奖励与每次动作的熵(entropy)

$$J(\pi) = \sum_{t\geq 0}[r_t + \alpha\mathcal{H}(\pi(\cdot|s_t))],$$

其中 α 是权重，随机变量 X 的熵 $\mathcal{H}(X)$ 的定义为

$$\mathcal{H}(X) = -\sum_{x\in\mathcal{X}}^{n} p(X=x)\log p(X=x)$$

$\mathcal{H}(X)$ 越大，表明随机变量 X 的概率分布“越分散”；反之，$\mathcal{H}(X)$ 越小，表明 X 的概率分布“越集中“，即确定性越大

最大化熵

- 给定状态 s，策略给出的动作 a 的概率分布“越随机”，其熵也就越大

$$J(\pi) = \sum_{t \geq 0} [r_t + \alpha \mathcal{H}(\pi(\cdot|s_t))],$$

同时考虑最大化奖励和熵，实质上是在鼓励策略进行探索：尽量不遗漏任何可能有用的动作

SAC

- SAC 算法用到了一个价值网络 $V(s;\psi)$，一个策略网络 $\pi(s;\phi)$ 和两个 Q 网络 $Q(s,a;\theta_1)$、$Q(s,a;\theta_2)$

  以上四个网络都使用梯度下降更新

  同时 SAC 还使用了一个目标价值网络 $V(s;\psi')$，使用 soft update 同步目标价值网络参数 ψ' 与价值网络参数 ψ

SAC 的价值网络

- 价值网络 $V(s,\psi)$ 的目标是最小化如下残差

$$J_V(\psi) = \mathbb{E}_s\left[(V(s;\psi) - \mathbb{E}_{a\sim\pi_\phi}[Q(s,a;\theta) - \log\pi(a|s;\phi)])^2\right],$$

其中 $\pi(a|s;\phi)$ 表示该策略下动作为 a 的概率；这个残差本身就考虑了最大化熵；这样的价值也称为 soft value

使用如下公式估计其梯度

$$\nabla_\psi J_V \approx \nabla_\psi V(s;\psi)[V(s;\psi) - Q(s,a;\theta) + \log\pi(a|s;\phi)]$$

SAC 的价值网络

- 通过随机梯度下降更新 ψ

$$\psi \leftarrow \psi - \eta_V \nabla_\psi J_V$$

更新 ψ 后，使用 soft update 更新 ψ'

$$\psi' \leftarrow \tau\psi + (1-\tau)\psi'$$

SAC 的 Q 网络

- Q 网络 $Q(s,a;\theta)$ 损失函数

$$L(\theta)=\mathbb{E}\left[(y-Q(s,a;\theta))^2\right],$$

其中目标

$$y=r+\gamma\mathbb{E}V(s';\psi')$$

使用如下公式计算其梯度

$$\nabla_\theta L\approx\nabla_\theta Q(s,a;\theta)[Q(s,a;\theta)-r-\gamma V(s';\psi')]$$

SAC 的 Q 网络

- SAC 使用两套独立的 Q 函数，使用时取其 Q 值较小者；对于高难度的任务，这种方法可以避免 Q 学习的“过估计”问题，效果优于使用一个 Q 函数

  通过随机梯度下降更新 θ_1 和 θ_2

$$\theta \leftarrow \theta - \eta_Q \nabla_\theta L$$

SAC 的策略网络

- SAC 的随机策略网络 $\pi(s;\phi)$ 可以改写为确定性的策略网络

$$a = f(\epsilon, s; \phi),$$

其中 $\epsilon \sim \mathcal{N}$ 是一个输入噪音向量，来自固定的概率分布（如高斯分布）$\mathcal{N}$

SAC 的策略网络

- 策略网络的目标是最小化

$$J_\pi(\phi) = \mathbb{E}_{s,\epsilon}\Big[\log \pi(f(\epsilon, s; \phi)|s; \phi) - Q(s, f(\epsilon, s; \phi); \theta)\Big]$$

使用如下公式估计其梯度

$$\nabla_\phi J_\pi \approx \nabla_\phi \log \pi(a|s; \phi) + (\nabla_a \log \pi(a|s; \phi) - \nabla_a Q(s, a)) \nabla_\phi f(\epsilon, s; \phi),$$

其中求梯度时 a 取值于 $f(\epsilon, s; \phi)$

SAC 的策略网络

- 通过随机梯度下降更新策略网络的参数 ϕ

$$\phi \leftarrow \phi - \eta_\pi \nabla_\phi J_\pi$$

SAC 算法

- 初始化参数：价值网络，目标价值网络，Q 网络，策略网络

Algorithm 1 Soft Actor-Critic

Initialize parameter vectors ψ, $\bar{\psi}$, θ, ϕ.
for each iteration **do**
  for each environment step **do**
    $\mathbf{a}_t \sim \pi_\phi(\mathbf{a}_t|\mathbf{s}_t)$
    $\mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$
    $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathbf{s}_t, \mathbf{a}_t, r(\mathbf{s}_t, \mathbf{a}_t), \mathbf{s}_{t+1})\}$
  end for
  for each gradient step **do**
    $\psi \leftarrow \psi - \lambda_V \hat{\nabla}_\psi J_V(\psi)$
    $\theta_i \leftarrow \theta_i - \lambda_Q \hat{\nabla}_{\theta_i} J_Q(\theta_i)$ for $i \in \{1, 2\}$
    $\phi \leftarrow \phi - \lambda_\pi \hat{\nabla}_\phi J_\pi(\phi)$
    $\bar{\psi} \leftarrow \tau\psi + (1-\tau)\bar{\psi}$
  end for
end for

SAC 算法

- 使用策略网络，生成轨迹，存入经验回放样本库

Algorithm 1 Soft Actor-Critic

Initialize parameter vectors ψ, $\bar{\psi}$, θ, ϕ.
for each iteration **do**
  for each environment step **do**
    $\mathbf{a}_t \sim \pi_\phi(\mathbf{a}_t|\mathbf{s}_t)$
    $\mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$
    $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathbf{s}_t, \mathbf{a}_t, r(\mathbf{s}_t, \mathbf{a}_t), \mathbf{s}_{t+1})\}$
  end for
  for each gradient step **do**
    $\psi \leftarrow \psi - \lambda_V \hat{\nabla}_\psi J_V(\psi)$
    $\theta_i \leftarrow \theta_i - \lambda_Q \hat{\nabla}_{\theta_i} J_Q(\theta_i)$ for $i \in \{1, 2\}$
    $\phi \leftarrow \phi - \lambda_\pi \hat{\nabla}_\phi J_\pi(\phi)$
    $\bar{\psi} \leftarrow \tau\psi + (1-\tau)\bar{\psi}$
  end for
end for

SAC 算法

- 使用梯度下降，更新价值网络，Q 网络，策略网络

Algorithm 1 Soft Actor-Critic

Initialize parameter vectors ψ, $\bar{\psi}$, θ, ϕ.
for each iteration **do**
  for each environment step **do**
    $\mathbf{a}_t \sim \pi_\phi(\mathbf{a}_t|\mathbf{s}_t)$
    $\mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$
    $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathbf{s}_t, \mathbf{a}_t, r(\mathbf{s}_t, \mathbf{a}_t), \mathbf{s}_{t+1})\}$
  end for
  for each gradient step **do**
    $\psi \leftarrow \psi - \lambda_V \hat{\nabla}_\psi J_V(\psi)$
    $\theta_i \leftarrow \theta_i - \lambda_Q \hat{\nabla}_{\theta_i} J_Q(\theta_i)$ for $i \in \{1, 2\}$
    $\phi \leftarrow \phi - \lambda_\pi \hat{\nabla}_\phi J_\pi(\phi)$
    $\bar{\psi} \leftarrow \tau\psi + (1-\tau)\bar{\psi}$
  end for
end for

SAC 算法

- Soft update 目标价值网络

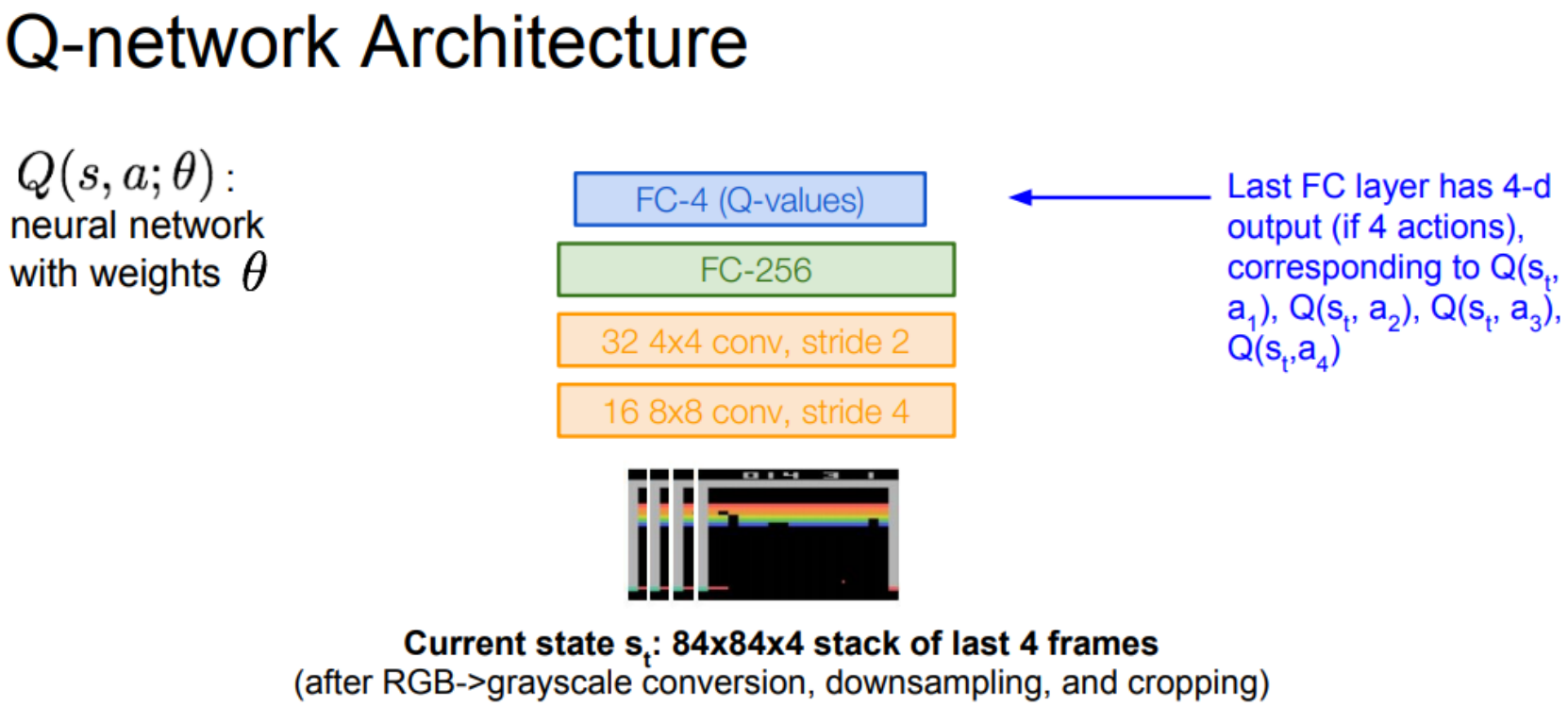
Algorithm 1 Soft Actor-Critic

Initialize parameter vectors ψ, $\bar{\psi}$, θ, ϕ.
for each iteration **do**
  for each environment step **do**
    $\mathbf{a}_t \sim \pi_\phi(\mathbf{a}_t|\mathbf{s}_t)$
    $\mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$
    $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathbf{s}_t, \mathbf{a}_t, r(\mathbf{s}_t, \mathbf{a}_t), \mathbf{s}_{t+1})\}$
  end for
  for each gradient step **do**
    $\psi \leftarrow \psi - \lambda_V \hat{\nabla}_\psi J_V(\psi)$
    $\theta_i \leftarrow \theta_i - \lambda_Q \hat{\nabla}_{\theta_i} J_Q(\theta_i)$ for $i \in \{1, 2\}$
    $\phi \leftarrow \phi - \lambda_\pi \hat{\nabla}_\phi J_\pi(\phi)$
    $\bar{\psi} \leftarrow \tau\psi + (1-\tau)\bar{\psi}$
  end for
end for

强化学习中的状态表示（state representation）

状态表示（state representation）

- **在玩 Atari 游戏的 DQN 中用了 4 帧图像信息作为状态 s**



状态表示（state representation）

- **状态表示：状态是什么？用怎样的数据表示**

- **强化学习中的状态表示，直接影响学习的效果**

状态需要满足马尔可夫条件

- 单帧图像能满足马尔可夫条件吗



状态表示最好尽量简洁、直观

- **状态表示方式的重要性**
  - 例子：Word Embedding，AlphaGo vs AlphaGO Zero
  - 状态表示简洁、直观时，需要大量算力和好的算法

Feature	# of planes	Description
Stone colour	3	Player stone / opponent stone / empty
Ones	1	A constant plane filled with 1
Turns since	8	How many turns since a move was played
Liberties	8	Number of liberties (empty adjacent points)
Capture size	8	How many opponent stones would be captured
Self-atari size	8	How many of own stones would be captured
Liberties after move	8	Number of liberties after this move is played
Ladder capture	1	Whether a move at this point is a successful ladder capture
Ladder escape	1	Whether a move at this point is a successful ladder escape
Sensibleness	1	Whether a move is legal and does not fill its own eyes
Zeros	1	A constant plane filled with 0
Player color	1	Whether current player is black

Neural Network Architecture The input to the policy network is a $19 \times 19 \times 48$ image stack consisting of 48 feature planes. The first hidden layer zero pads the input into a 23×23 image,

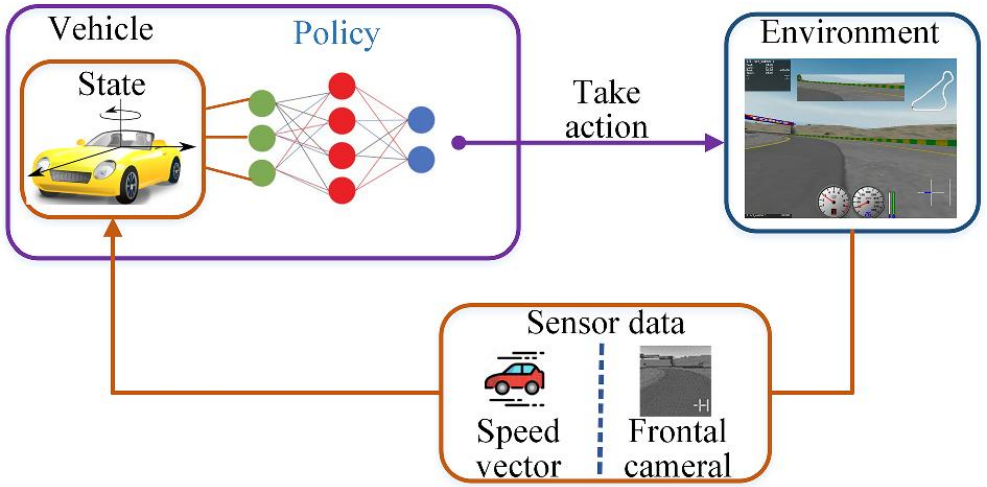
AlphaGo

Neural network architecture. The input to the neural network is a $19 \times 19 \times 17$ image stack comprising 17 binary feature planes. Eight feature planes, X_t, consist of binary values indicating the presence of the current player's stones ($X_t^i = 1$ if intersection i contains a stone of the player's colour at time-step t; 0 if the intersection is empty, contains an opponent stone, or if $t < 0$). A further 8 feature planes, Y_t, represent the corresponding features for the opponent's stones. The final feature plane, C, represents the colour to play, and has a constant value of either 1 if black is to play or 0 if white is to play. These planes are concatenated together to give input features $s_t = [X_t, Y_t, X_{t-1}, Y_{t-1}, \ldots, X_{t-7}, Y_{t-7}, C]$. History features X_t, Y_t are

AlphaGo Zero

状态表示需要尽量简洁、直观

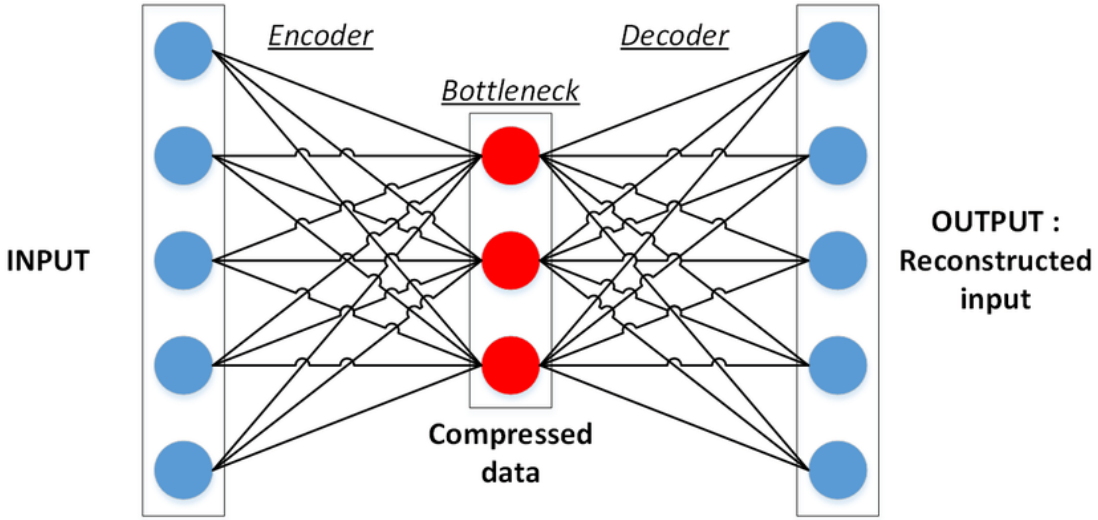
- **但如果算力足够，训练方法足够好，也可以忽视状态表示，直接做端到端（end to end）训练**
  - 将传感器的信号当作状态（输入端）
  - 将执行器的动作当作输出（输出端）



- **目前条件下，对复杂问题，端到端强化学习的方法通常效果一般**

状态表示学习

- 可以通过机器学习的方法，得到最佳的状态表示



练习题

一般认为，以下强化学习算法中最适合用于连续系统控制的是_____

A. Deep Q-Learning（DQN）

B. Double Deep Q-Learning（DDQN）

C. Deep Deterministic Policy Gradient（DDPG）

D. Asynchronous Advantage Actor Critic（A3C）

练习题

一般认为，以下强化学习算法中最适合用于连续系统控制的是_____

A. Deep Q-Learning（DQN）

B. Double Deep Q-Learning（DDQN）

C. Deep Deterministic Policy Gradient（DDPG）

D. Asynchronous Advantage Actor Critic（A3C）

【答案】：C

【解析】：DDPG专为连续动作空间设计，采用Actor-Critic架构。

Actor网络直接输出连续动作值，Critic网络评估动作的Q值，

结合确定性策略梯度，是连续系统控制的经典算法，最适合连续系统控制。